Killer CLIs for Platform Teams

Designing Tools Developers Love

By Chaitanya Rahalkar

Little About Me

Software Security Engineer

BLOCK

Also building OmniChat AI!

- Cybersecurity, AI and Systems
  Avid CTF Player
- Fuzzing, Reverse Engineering, AppSec
  Linux, Generative AI
- Anything, AI or Cybersecurity
- Go Jackets!

Why CLIs Matter?

- Developers often prefer command-line tools for speed and control. CLIs fit into the environments (terminals, editors) they already use.
- A good CLI can replace multiple GUI steps with one command, reducing context switches

Why Developers Love Great CLIs (Psychology)

- ***Efficiency***: One command can replace many web clicks or GUI actions.
- ***Low Cognitive Load***: CLIs limit choices; devs think in "*verbs*" and "commands". A GUI can present thousands of options, but a CLI focuses on one action at a time.
- ***Scriptability***: CLIs are easy to script and automate (piping output, combining commands).
- ***Control & Feedback***: Developers appreciate precise control and clear, immediate feedback in the terminal.

Principle 1 – Intelligent Defaults & Automation

- ***Sensible Defaults***: Choose defaults that cover the common case. If no arguments are given, run the primary action or show help.
- ***Context Awareness***: Detect context automatically (e.g., infer repository or project from current directory). Link project metadata behind the scenes.
- ***Automate Boilerplate***: Automate repetitive setup tasks (scaffolding, configs, auth). The CLI's goal is to make "getting started" fast and easy.
- Example – ***GitHub CLI***: Instead of clicking through GitHub UI to open an issue, one command does it without leaving the terminal.

Principle 2 – Progressive Disclosure & Discoverability

- ***Keep the Base Simple***: Start with minimal required flags/arguments. For new users, do as much automatically as possible.

- ***Reveal Power Gradually***: Provide subcommands or advanced flags for power users to uncover when needed. Think of git – git commit does basic work, but git commit --amend is there if you want it.

- ***Help & Documentation***: Always include clear *--help* output, examples, and suggestions. Prefer to guide rather than overwhelm.

- Example: Show a brief help by default, and let users drill down. (Shopify's design wisdom: "*Make the initial interface simple, but provide access to advanced functionality for those who need it.*"

Principle 3 – Clear Communication & Helpful Errors

- ***Concise Success Messages***: On success, often the best practice is no message at all. Exit quietly (Unix style) unless you have something important to say.

- ***Verbose Errors***: If something fails, provide detailed, actionable output. Show why it failed and how to fix it. (“Bad news shouts”)

- ***Suggest Corrections***: If a command or flag is unrecognized, show alternatives. E.g., npm suggests “*Did you mean?*” when you typo a script.

- ***Active, Developer-Friendly Language***: Use the active voice and imperative (verb-first) for commands (e.g., create project, not project create).

Principle 4 – Balance Power and Simplicity

- Design for Two Audiences: Newcomers need simplicity and guidance; power users need flexibility and speed.
- Expose “Expert Modes”: Allow advanced usage (raw flags, config files, plugin hooks) for those who want full control. Hide them from novices unless requested.
- Follow Conventions: If UNIX convention suggests service start instead of start service, honor it; users already have mental models shopify.com.
- Learnable Defaults: Make the easy path easy, but don’t lock out the expert path. For example, offer a sane default and an --advanced flag or step-by-step mode.

Example – GitHub CLI (gh)

- ***Streamlines GitHub tasks***: commands for repos, issues, PRs, actions, etc. No need to switch to web UI each time.

- ***Workflow Example***: Without CLI, creating an issue = open browser, navigate repo, click “Issues” → “New Issue”. With gh issue create, do it in one step in terminal geeksforgeeks.org.

- ***Context Integration***: Operates in your git repo (no need to specify owner/repo every time). Follows active voice in commands (repo create, pr merge).

- ***Outcome***: Developers report huge time savings because they stay in the terminal for end-to-end GitHub workflows.

Example – GitHub CLI (gh)

```
~/cli/cli$ gh pr status

Relevant pull requests in cli/cli

Current branch
  There is no pull request associated with [master]

Created by you
  #255  Update docs [docs]
   - Changes requested - Checks passing

Requesting a code review from you
  You have no pull requests to review
```

Example – Modernizing an Internal CLI

- ***Before***: Developers used scattered scripts or ad-hoc CLIs with inconsistent naming, manual config, poor help text.

- ***After***: Replace with a unified CLI using a framework (e.g., Cobra, Oclif, Clap). Standardize commands (tool build, tool deploy, etc.).

- ***Improvements***: Add *--help* for each command, clear error messages, and features like tab completion. Use config files or prompts to remember settings.

- ***Impact***: Adoption rises dramatically. Developers choose the new CLI over homegrown scripts because it's easier to learn and use day-to-day.

Tips for Building “Killer” CLIs Today

- ***Use a CLI Framework***: Leverage tools like Cobra (Go), Oclif (TypeScript), Click (Python), or Rust’s Clap. They handle parsing, help pages, and sometimes shell completion out of the box.

- ***Write Good Help and Docs***: Every command should have examples. Maintain a short README or --guide section. Consider embedding quickstarts or code snippets.

- ***Support Autocomplete***: Many CLI libraries can auto-generate shell completion scripts. This makes using flags and commands much faster.

- ***Iterate with Users***: Solicit feedback from the team. Watch people use the CLI or survey them: *What’s confusing? Which commands are missing?* Prioritize fixes.

- ***Consistent Style:*** Align terminology with your ecosystem. For instance, if your platform is Kubernetes-based, use terms familiar to SREs.

Summary & Actionable Takeaways

- ***Apply These Principles***: Intelligent defaults, progressive disclosure, clear messaging, and balanced power are core to anyCLI you build.

- ***Audit Your CLIs***: Pick one of your internal tools now. Does it follow these rules? How could it be simpler or more helpful?

- ***Empower Developers***: A "***killer CLI***" isn't just a nicer interface – it lets devs iterate and ship faster with less friction. That's the ultimate goal.

- ***Next Steps***: Commit to making even one improvement (e.g., add a missing --help, simplify an option, or show a better error). Small wins build momentum.

Thank You!

Let us chat: c@rahalkar.dev

chairahalkar

/in/chaitanyarahalkar

Get the slides: bit.ly/platform-con-2025